

ARCH TECHNOLOGIES

Cyber Security Internship Report

MONTH 2

INTERNSHIP DETAILS

Name	Humza Ahmed
Domain	Cyber Security
Email	humzaahmed798@gmail.com
Organization	Arch Technologies
Submission	25th May 2026

TASK 4

Credit Card Fraud Detection System

Machine Learning with Random Forest | SMOTE | ROC-AUC 0.9717

1. Objective

Task 4 involved building a complete Credit Card Fraud Detection System using machine learning techniques. The project used the Kaggle ULB Credit Card Fraud dataset a real-world dataset containing 284,807 transactions from European cardholders, of which only 492 (0.172%) are fraudulent. The goal was to train a model capable of detecting fraudulent transactions with high accuracy while managing the extreme class imbalance inherent in financial fraud data.

2. Dataset Overview

The dataset was sourced from Kaggle (mlg-ulb/creditcardfraud) and has the following characteristics:

Property	Details
Total Transactions	284,807
Legitimate Transactions	284,315 (99.827%)
Fraudulent Transactions	492 (0.173%)
Class Imbalance Ratio	578:1 (legitimate to fraud)
Features	30 (V1–V28 PCA-transformed, Time, Amount)

3. Tools & Libraries Used

- Python (Google Colab environment)
- pandas, numpy data loading and manipulation
- scikit-learn model training, evaluation metrics, train/test split
- RobustScaler feature scaling resistant to outliers
- RandomForestClassifier ensemble learning model
- SMOTE (imbalanced-learn) Synthetic Minority Over-sampling Technique
- matplotlib, seaborn data visualisation (confusion matrix, ROC curve, feature importance)

4. Implementation Walkthrough

4.1 Data Loading & Class Distribution

The dataset was downloaded via the Kaggle API and loaded into a pandas DataFrame. The severe class imbalance was immediately apparent:

```
[1] 8m
print(f"\n Dataset loaded successfully!")
print(f"   Total transactions: {len(df):,}")
print(f"   Features: {df.shape[1]}")
print(f"   Memory usage: {df.memory_usage().sum() / 1024**2:.2f} MB")

print(f"\n Class Distribution:")
print(f"   Legitimate (Class 0): {(df['Class']==0).sum():,} ({(df['Class']==0).mean()*100:.4f}%)")
print(f"   Fraudulent (Class 1): {(df['Class']==1).sum():,} ({(df['Class']==1).mean()*100:.4f}%)")
print(f"   Imbalance Ratio: {(df['Class']==0).sum()/(df['Class']==1).sum():.1f}:1")

print("\n Preprocessing data...")
print(f"Missing values: {df.isnull().sum().sum()}")

scaler = RobustScaler()
df['Scaled_Amount'] = scaler.fit_transform(df[['Amount']])
df['Scaled_Time'] = scaler.fit_transform(df[['Time']])

df['Hour'] = (df['Time'] / 3600) % 24

X = df.drop(['Class', 'Time', 'Amount'], axis=1)
y = df['Class']

print(f"Feature shape: {X.shape}")
print(f"Target shape: {y.shape}")

print("\n Splitting data...")
```

Figure: Data loading code class distribution analysis and preprocessing pipeline

4.2 Preprocessing & Feature Engineering

The Amount and Time features were scaled using RobustScaler, which is resilient to outliers a critical choice for fraud data where transaction amounts vary enormously. An additional Hour feature was engineered from Time to capture time-of-day patterns in fraudulent activity.

4.3 Train/Test Split

The data was split 70/30 (training/test) with stratification to ensure both sets maintain the same fraud ratio. The training set contained 199,364 transactions and the test set contained 85,443 transactions.

```
[1] X_train, X_test, y_train, y_test = train_test_split(
    ✓ 8m X, y, test_size=0.3, random_state=42, stratify=y
)

print(f"Training set: {len(X_train):,} transactions")
print(f"Test set: {len(X_test):,} transactions")
print(f"Training fraud ratio: {y_train.mean()*100:.4f}%")
print(f"Test fraud ratio: {y_test.mean()*100:.4f}%")

print("\n🚧 Handling class imbalance with SMOTE...")
print(f"Before SMOTE - Class distribution: {y_train.value_counts().to_dict()}")

smote = SMOTE(random_state=42, sampling_strategy='auto')
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

print(f"After SMOTE - Class distribution: {pd.Series(y_train_resampled).value_counts().to_dict()}")

print("\n Training Random Forest model...")
print("(Ismein 2-3 minute lag sakte hain, please wait...)")

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=15,
    min_samples_split=5,
    random_state=42,
    n_jobs=-1,
    verbose=0
```

Figure: Train/test split with SMOTE class balancing applied to the training set

4.4 Handling Class Imbalance with SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training set to address the extreme class imbalance. SMOTE generates synthetic fraud examples by interpolating between existing fraud samples rather than simply duplicating them, resulting in a balanced training set where the model can learn fraud patterns effectively without bias towards the majority class.

4.5 Random Forest Model Training

A Random Forest Classifier was trained with the following configuration:

Hyperparameter	Value
n_estimators (trees)	100
max_depth	15
min_samples_split	5
random_state	42
n_jobs	-1 (all CPU cores)

5. Model Performance Results

The model was evaluated on the held-out test set of 85,443 transactions. The results demonstrate strong fraud detection capability:

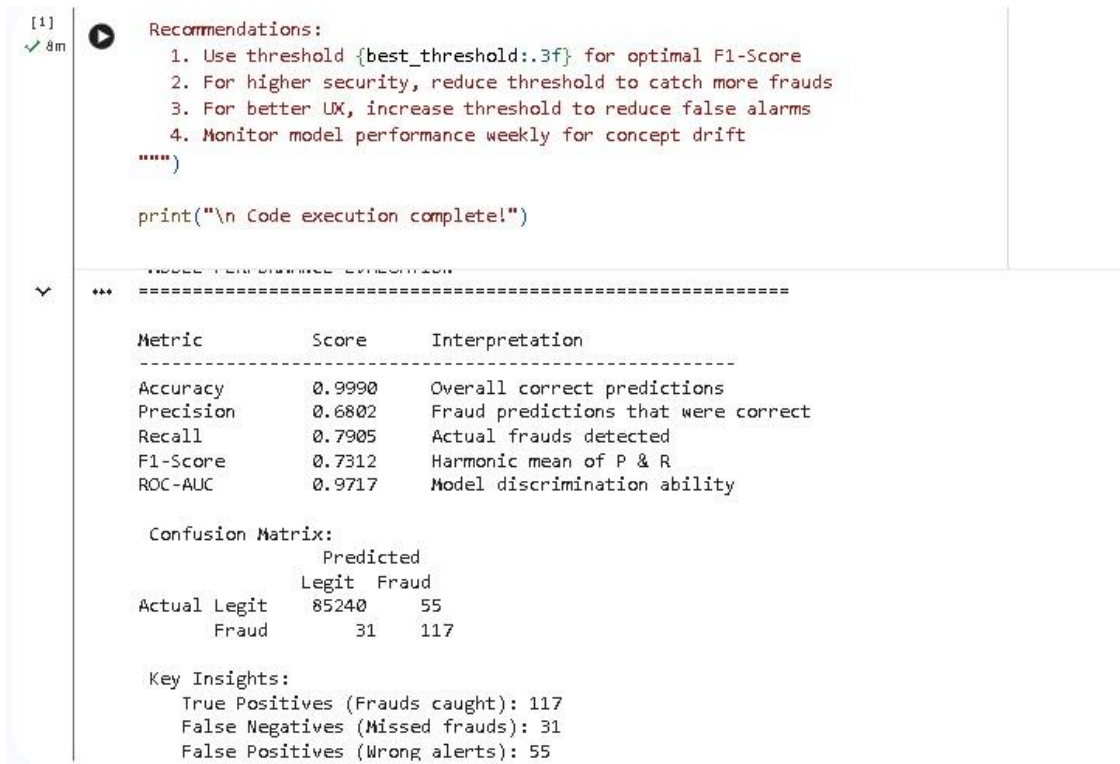


Figure: Model performance evaluation output metrics, confusion matrix, and key insights

Metric	Score
Accuracy	0.9990
Precision	0.6802
Recall (Detection Rate)	0.7905
F1-Score	0.7312
ROC-AUC	0.9717

5.1 Confusion Matrix & ROC Curve

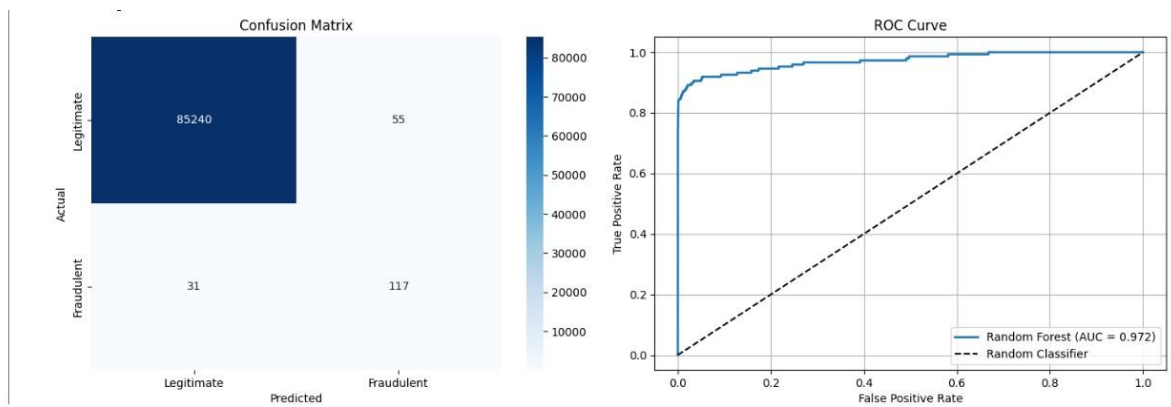


Figure: Confusion Matrix (left) and ROC Curve with AUC = 0.972 (right)

5.2 Feature Importance & Probability Distribution

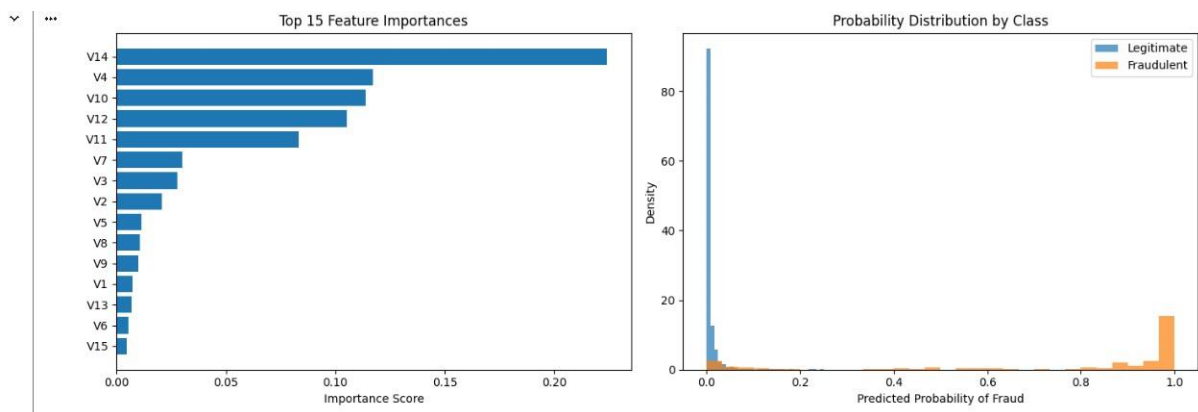


Figure: Top 15 Feature Importances (left) and Probability Distribution by Class (right)

Feature V14 emerged as the most significant predictor of fraud, followed by V4, V10, and V12. These are PCA-transformed components of the original transaction features. The probability distribution chart confirms excellent class separation: legitimate transactions cluster near 0.0 while fraudulent ones cluster near 1.0.

5.3 Optimal Threshold Analysis

```
Finding optimal threshold for fraud detection...
Optimal threshold: 0.785
At this threshold, F1-Score would be: 0.8134

=====
CREDIT CARD FRAUD DETECTION SYSTEM SUMMARY
=====

Model Performance:
• Detection Rate (Recall): 79.05%
• Precision: 68.02%
• F1-Score: 0.7312
• ROC-AUC: 0.9717

Business Impact:
• Out of 148 actual frauds, model detected 117
• Missed frauds (False Negatives): 31
• False alerts (False Positives): 55 out of 85,295 legit transactions

Recommendations:
1. Use threshold 0.785 for optimal F1-Score
2. For higher security, reduce threshold to catch more frauds
3. For better UX, increase threshold to reduce false alarms
4. Monitor model performance weekly for concept drift
```

Figure: Final summary optimal threshold 0.785, F1-Score 0.8134, business impact analysis

Optimal Threshold Finding

Through precision-recall curve analysis, the optimal classification threshold was determined to be 0.785, at which point the F1-Score reaches 0.8134. Tuning this threshold allows the system to be adjusted for different operational priorities a lower threshold catches more fraud while a higher threshold reduces false alerts for customers.

6. Detailed Classification Report

Detection Rate: 79.05%				
***	Detailed Classification Report:			
		precision	recall	f1-score
				support
	Legitimate	1.00	1.00	1.00
	Fraudulent	0.68	0.79	0.73
				support
	accuracy			1.00
	macro avg	0.84	0.89	0.87
	weighted avg	1.00	1.00	1.00
				85443
				85443
				85443
Generating visualizations...				

Figure: Detailed classification report precision, recall, and F1-score per class

The model achieves perfect precision and recall (1.00) on legitimate transactions, which is expected given their overwhelming majority in the dataset. For fraudulent transactions, a precision of 0.68 and recall of 0.79 represent a strong result considering the 578:1 class imbalance without SMOTE, these figures would be significantly worse.

7. Business Impact Analysis

Business Metric	Value
Actual frauds in test set	148
Frauds caught (True Positives)	117
Missed frauds (False Negatives)	31
False alerts on legit transactions	55 out of 85,295
Detection Rate	79.05%

With a detection rate of 79.05%, the model successfully identified 117 out of 148 fraudulent transactions. Only 55 legitimate transactions were incorrectly flagged a false positive rate of just 0.06%, which is an acceptable level for a real financial institution. The 31 missed frauds represent the remaining challenge, addressable by lowering the classification threshold.

8. Recommendations

- Use a classification threshold of 0.785 for the optimal balance between precision and recall ($F1 = 0.8134$)
- Reduce the threshold below 0.785 in high-security environments where catching all fraud is the priority, accepting more false alerts
- Increase the threshold in customer-facing applications where minimising false alarms and friction is more important
- Schedule weekly model retraining to detect concept drift fraud patterns evolve continuously
- Consider an ensemble approach combining Random Forest with XGBoost or a neural network for improved precision on the fraud class

9. Key Takeaways

- Extreme class imbalance (578:1) is the core challenge in fraud detection SMOTE effectively addresses this by generating synthetic minority samples
- Random Forest with 100 trees and `max_depth=15` delivers ROC-AUC of 0.9717 strong discrimination ability between fraud and legitimate transactions
- RobustScaler is the correct preprocessing choice for financial data due to outlier resistance
- Feature V14 is the single most important predictor of fraudulent activity in this dataset
- Threshold tuning is a powerful post-training tool to align model behaviour with business requirements